Reinforcement Learning Foundations

Software Engineering Paradigms

At the heart of software engineering: processing and moving data.

To process data, you can write programs manually or use machine learning.
When should one use each?

RULE-BASED

You have data, and you want to process it so you get a **given set of outputs**.



You write rules by hand.

LEARNING-BASED

You have data and a given set of outputs, and you want to process it so you get a **given set of rules**.



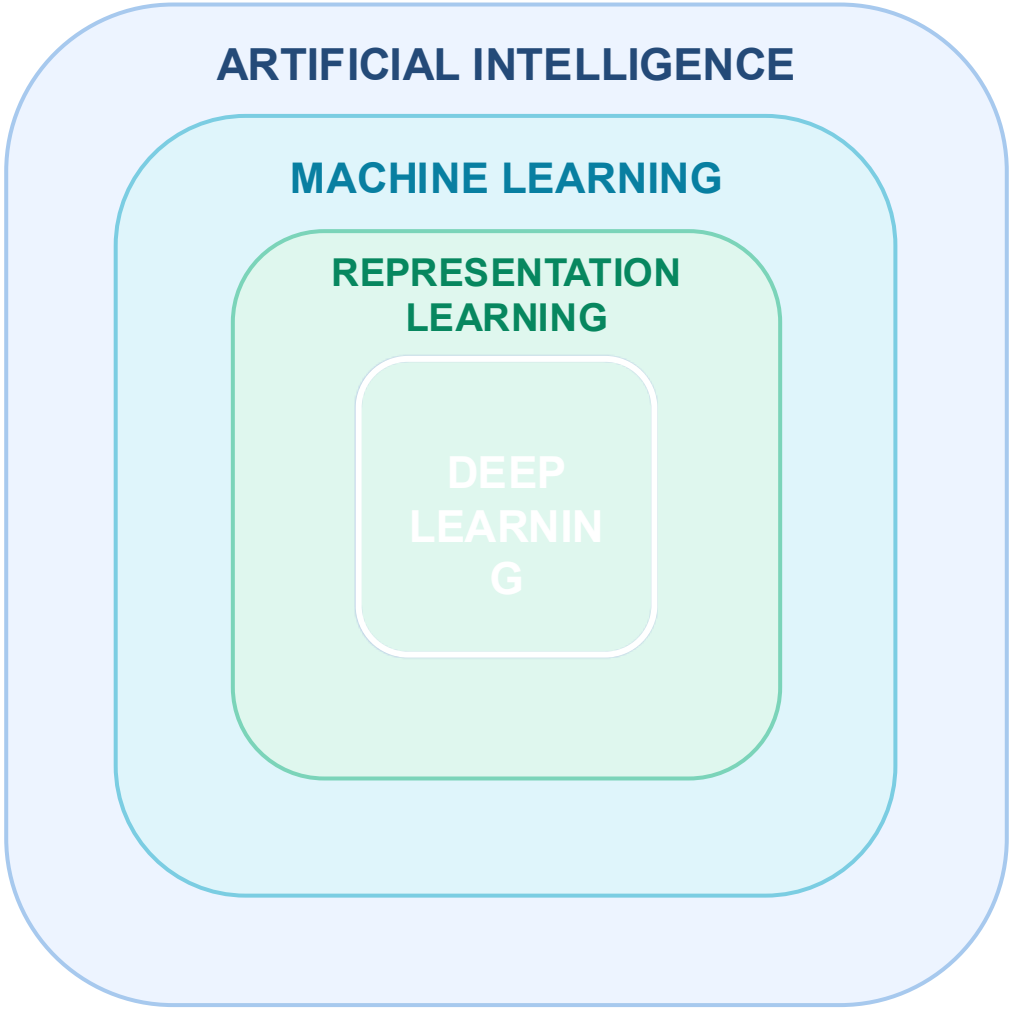
The rules are learned by an algorithm.

DEFINITION OF MACHINE LEARNING

- Machine learning is discovering a set of **parameters** that seems consistent with the statistics of the training data.
  Any algorithm that allows this is considered machine learning.
- It allows you to discover and parameterize different levels of **statistical patterns**.

From Artificial Intelligence to Deep Learning

Learning from data



Artificial Intelligence
The broad field of systems that perform tasks associated with intelligent behavior.

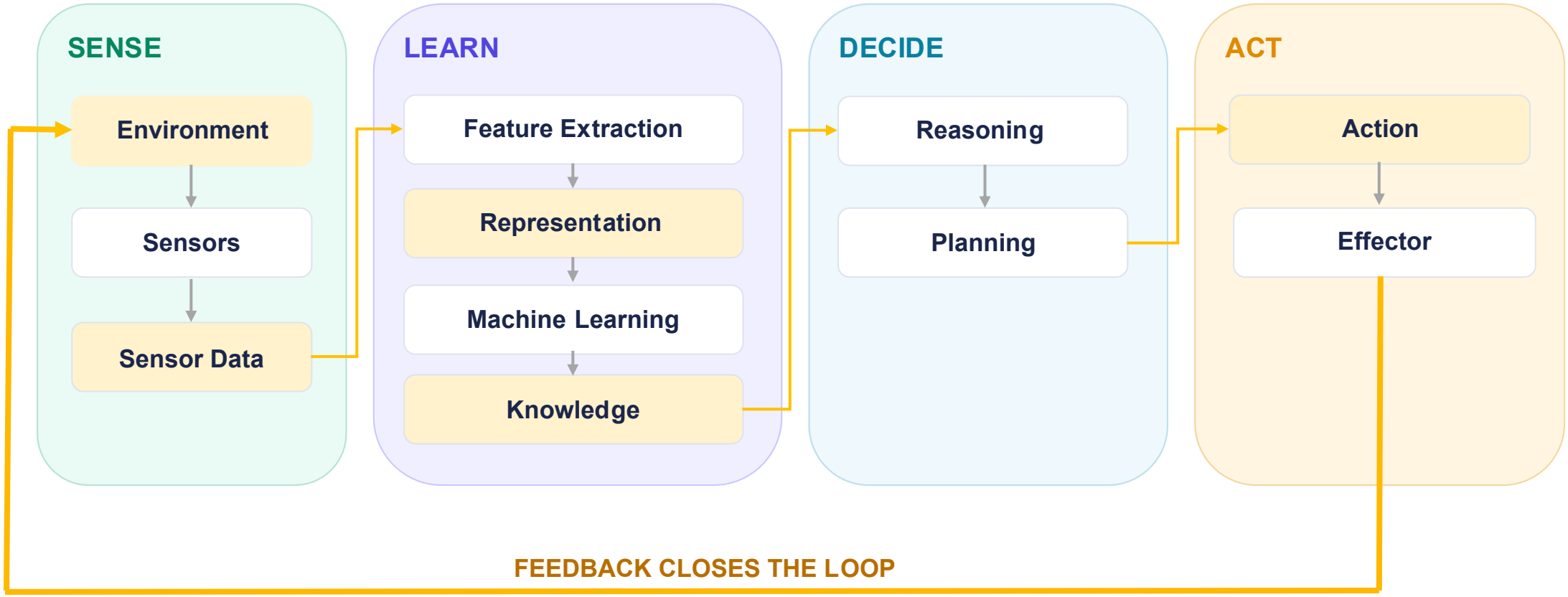
Machine Learning
Machine learning is discovering a set of parameters that seems consistent with the statistics of the training data.

Representation Learning
Representation learning allows a system to discover useful features or representations from data instead of relying entirely on manually designed features.

Deep Learning
Deep learning uses multiple computational layers to learn increasingly abstract representations from data.

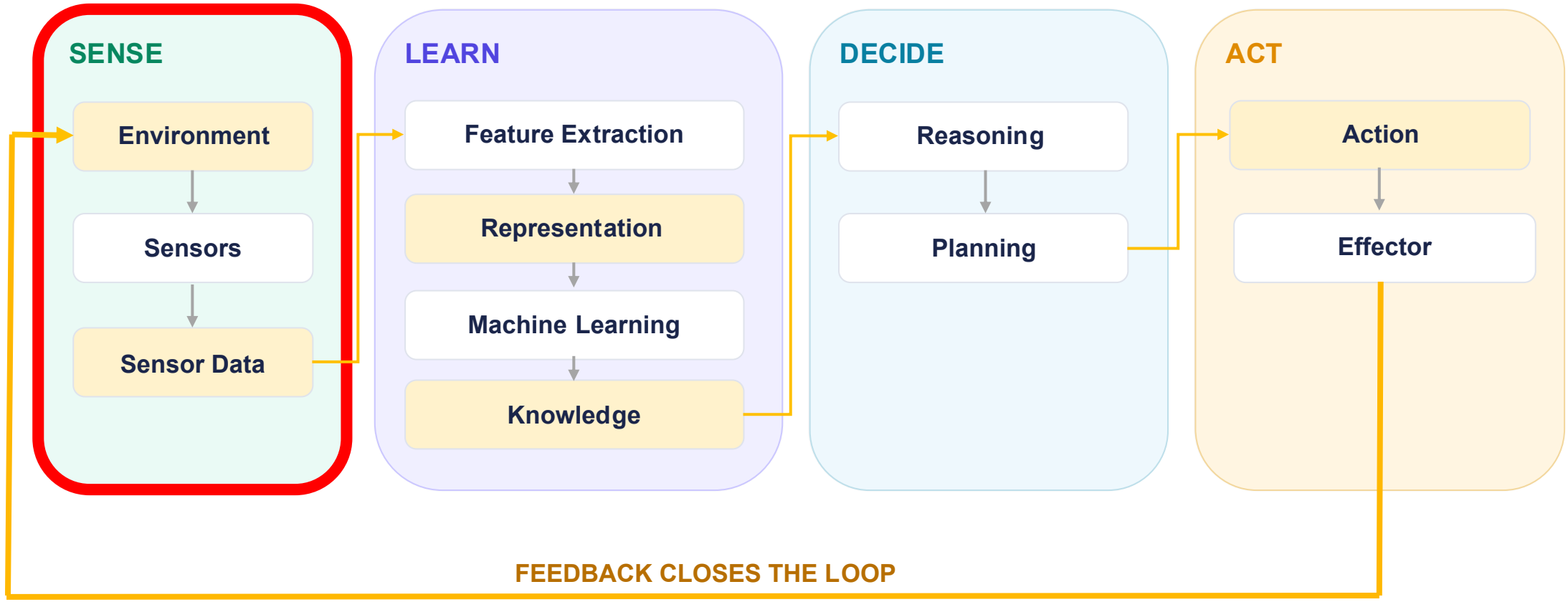
What can be learned from data?

Signals become representations, learned knowledge, plans, and finally actions that change the environment.



What can be learned from data?

Signals become representations, learned knowledge, plans, and finally actions that change the environment.



Sensing the Environment



LiDAR



Camera



Radar



Stereo Camera



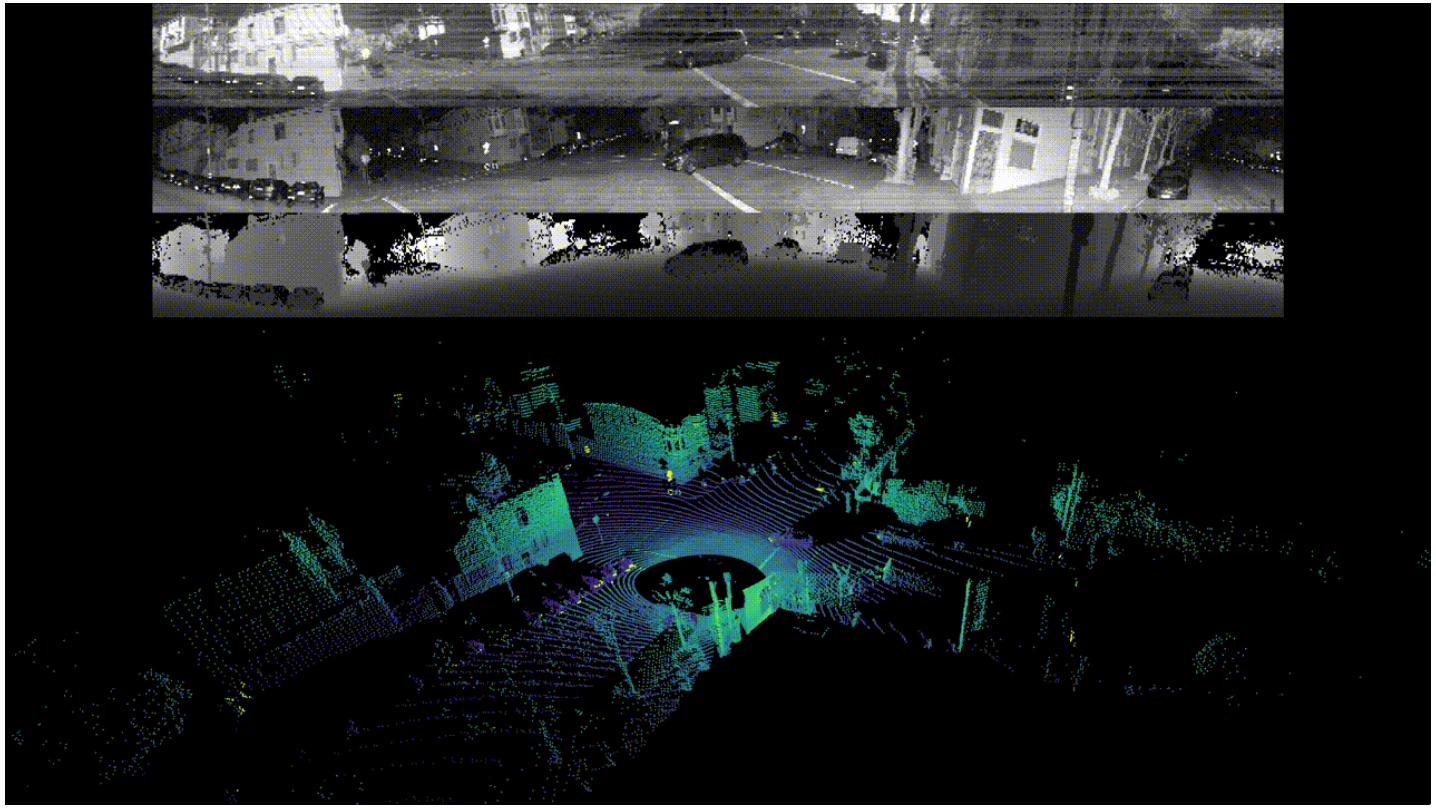
Microphone



GNSS / GPS

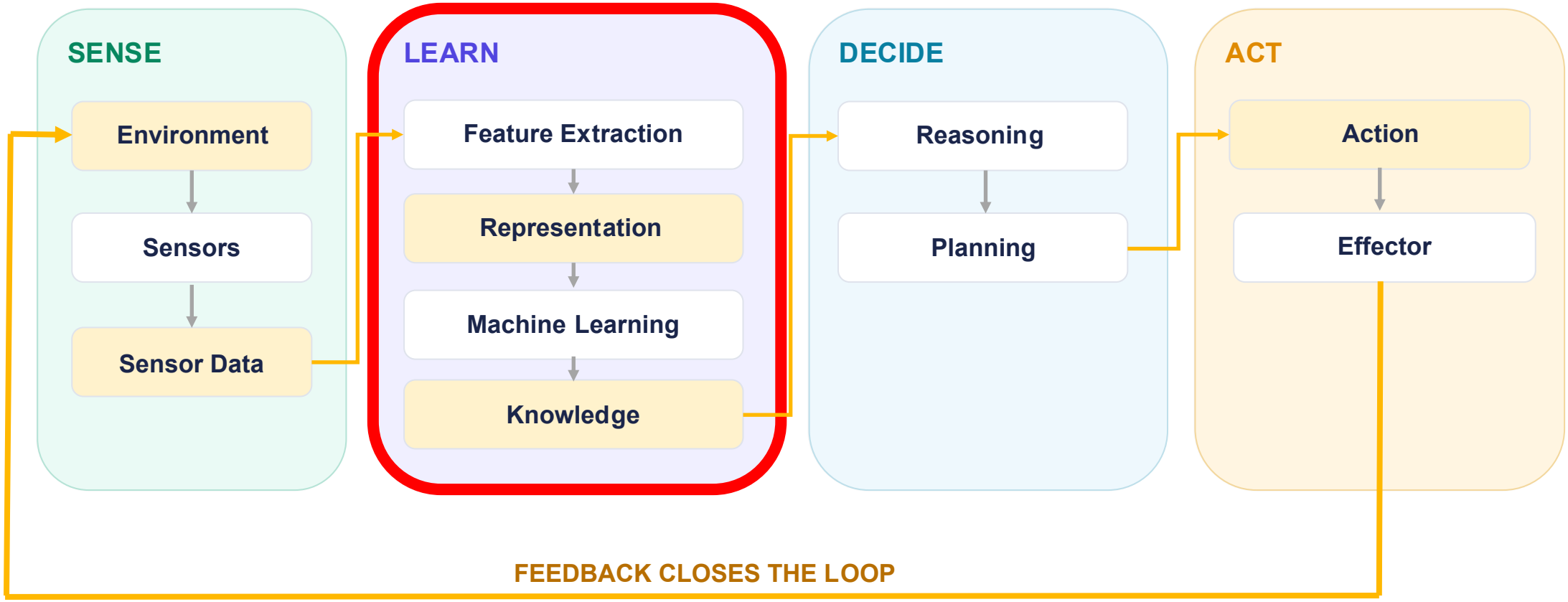


IMU

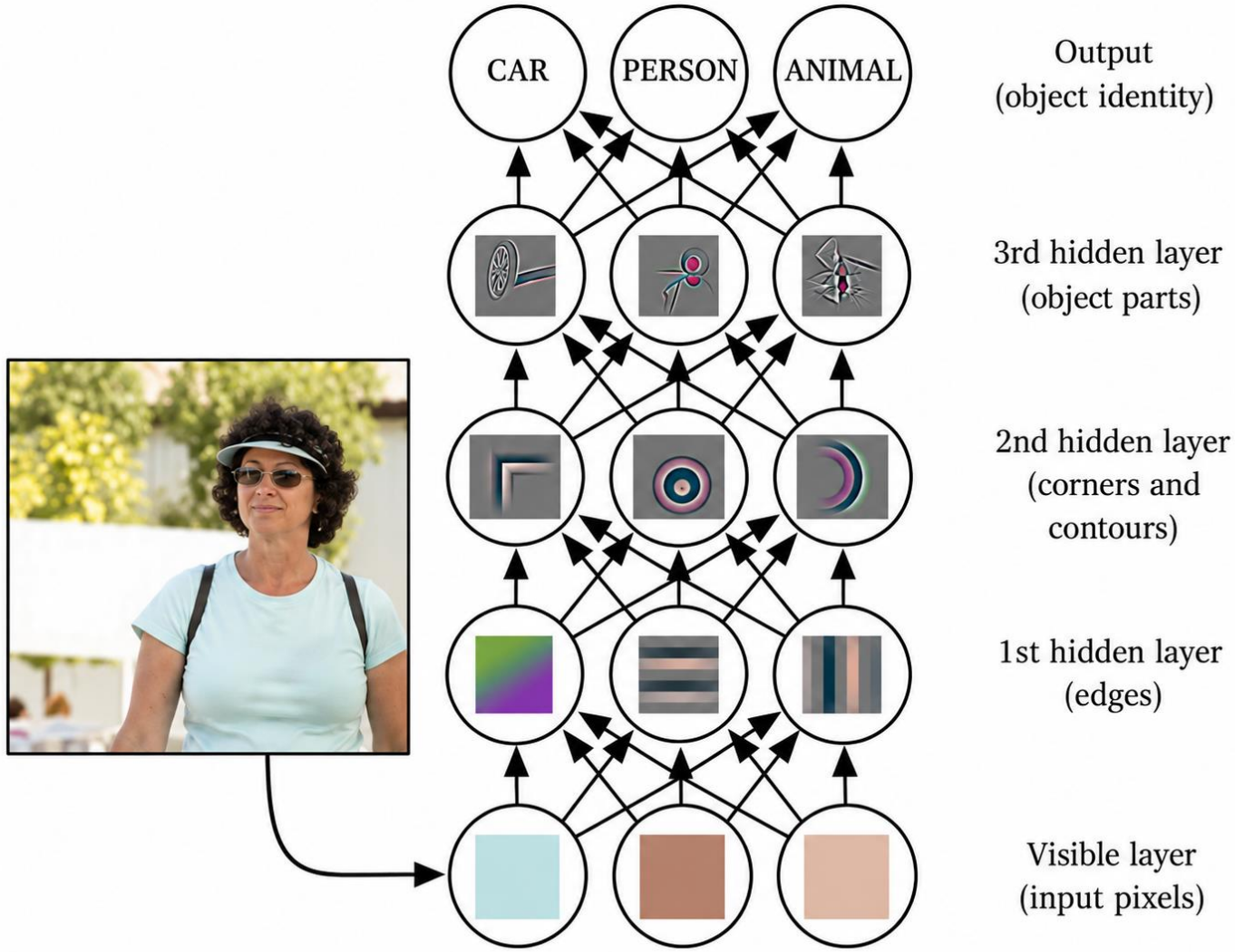


What can be learned from data?

Signals become representations, learned knowledge, plans, and finally actions that change the environment.

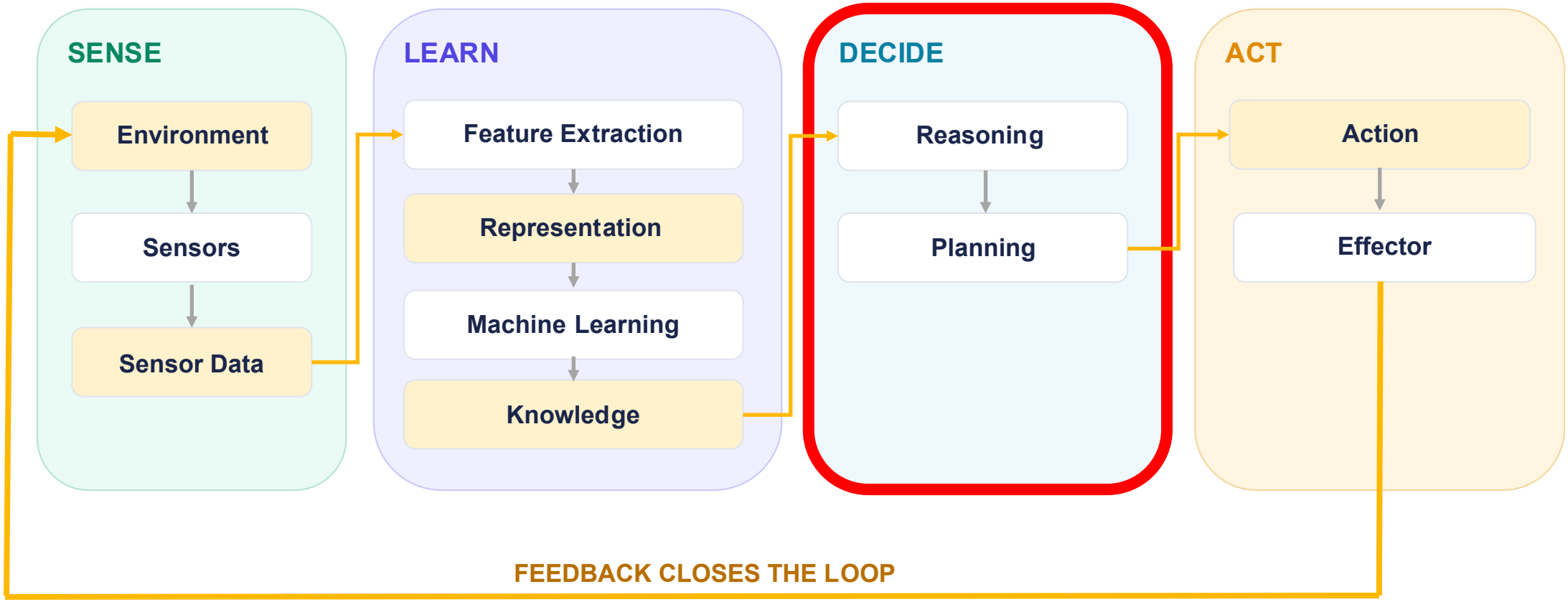


Forming Representations



What can be learned from data?

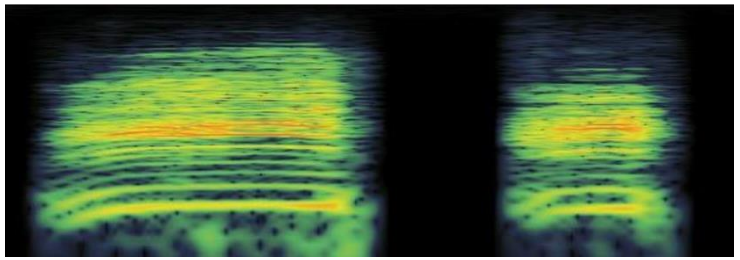
Signals become representations, learned knowledge, plans, and finally actions that change the environment.



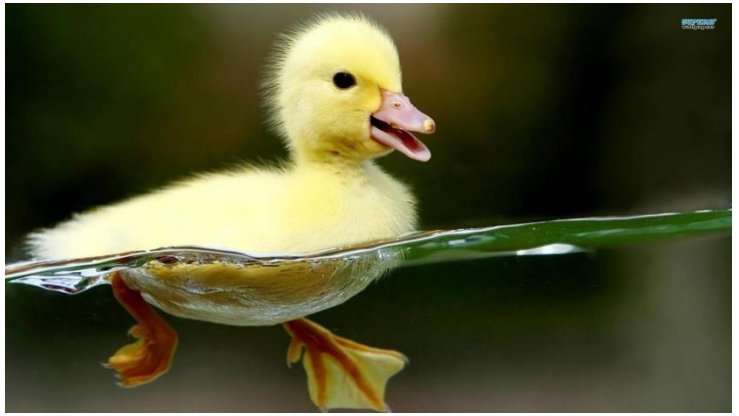
Aggregating Information



Image Recognition:
If it looks like a duck



Audio Recognition:
Quacks like a duck



Activity Recognition:
Swims like a duck

Reasoning

"If I turn a glass upside down on a table, and then place a coin on top of the upside-down glass, where is the coin in relation to the inside of the glass?"

Visualized spatial orientation of coin relative to glass cavity

So the coin is sitting on top of the glass base's exterior surface, which is now facing upward since the glass has been flipped over. The hollow cavity inside the glass is trapped underneath, pressed against the table by the rim, so the coin isn't inside that cavity at all—it's resting on the outside of the base.

Done

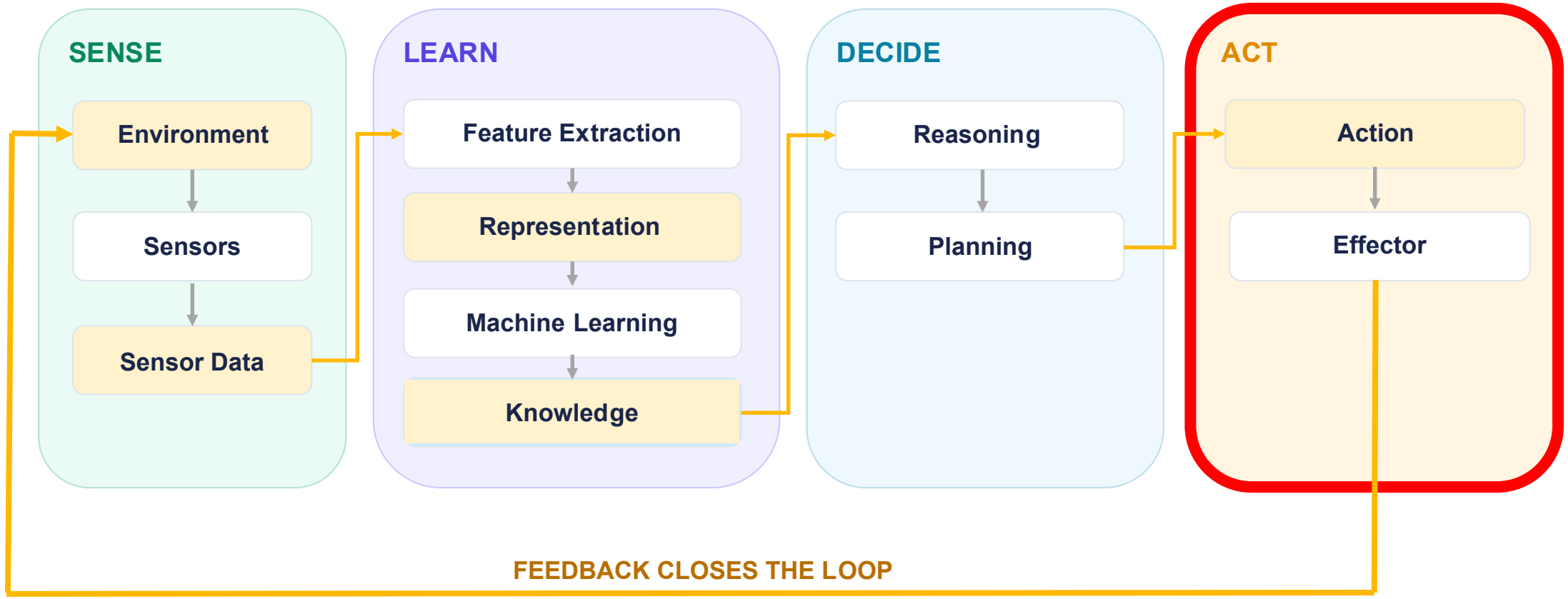
The coin is outside the glass, not inside it at all.

When you flip the glass upside down, the open end (where you'd normally pour liquid in) faces down toward the table, and the closed base faces up. The "inside" — the hollow, concave cavity — is now sealed between the rim and the table, facing downward.

The coin, resting on top, sits on the *outside* of the base — the surface that's normally the bottom of the glass when it's right-side up. There's no path from where the coin sits to the interior cavity; the glass material itself separates them.

What can be learned from data?

Signals become representations, learned knowledge, plans, and finally actions that change the environment.



The Three Learning Paradigms



Supervised Learning

- Learns from **input–target pairs**
- The target specifies the desired output
- Objective: minimize prediction error
- Examples: classification, regression



Unsupervised Learning

- Learns from **unlabelled data**
- Discovers structure, regularities, or useful representations
- No predefined correct output is provided
- Examples: clustering, dimensionality reduction



Reinforcement Learning

- Learns through **interaction**
- Actions affect future states and rewards
- Feedback evaluates behaviour, not individual correct answers
- Objective: maximize long-term reward

Supervised learning is instructive: here's the right answer. Reinforcement learning is evaluative: here's how good it was.

What is Reinforcement Learning?



Reinforcement Learning



ENVIRONMENT

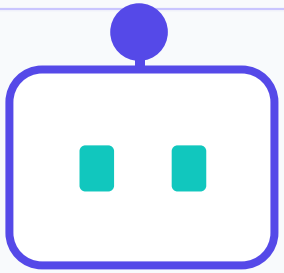
The **external world** whose **state changes** in response to actions.

ACTION

At each step, the agent:

- **Executes action**
- **Observes new state**
- **Receives reward**

STATE, REWARD



AGENT

The learner that **maps** observations to decisions.

Environments

Observability

- **Fully observable:** the agent receives all relevant information
- **Partially observable:** some information is hidden or must be inferred

Agents

- **Single-agent:** one agent acts in the environment
- **Multi-agent:** several agents cooperate or compete

Uncertainty

- **Deterministic:** the same action in the same state gives the same result
- **Stochastic:** outcomes may vary

Timing

- **Turn-based:** the environment waits for the agent
- **Dynamic:** the environment continues changing while the agent acts

State and Action Spaces

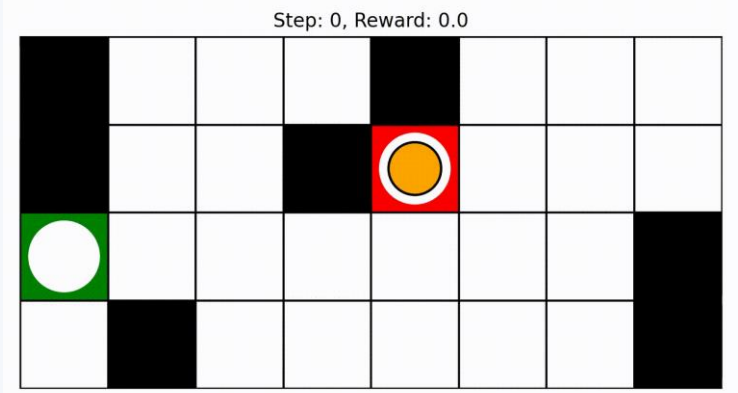
- **Discrete:** a limited set of states or actions
- **Continuous:** states or actions take numerical values
- **Hybrid:** discrete decisions combined with continuous control

Bottom line **These dimensions determine what the agent can observe, what it can do, and how difficult learning becomes.**

REINFORCEMENT LEARNING EXAMPLE

Grid World

An agent moves through a small grid and tries to reach a goal.



RL FORMULATION

STATE	the agent’s current cell
ACTIONS	up, down, left, right
REWARD	goal reward and possible movement penalties
OBJECTIVE	reach the goal efficiently

ENVIRONMENT

FULLY OBSERVABLE | SINGLE-AGENT | DISCRETE | DET. / STOCHASTIC | EPISODIC

MAIN LESSON: The complete MDP can be seen and represented explicitly.

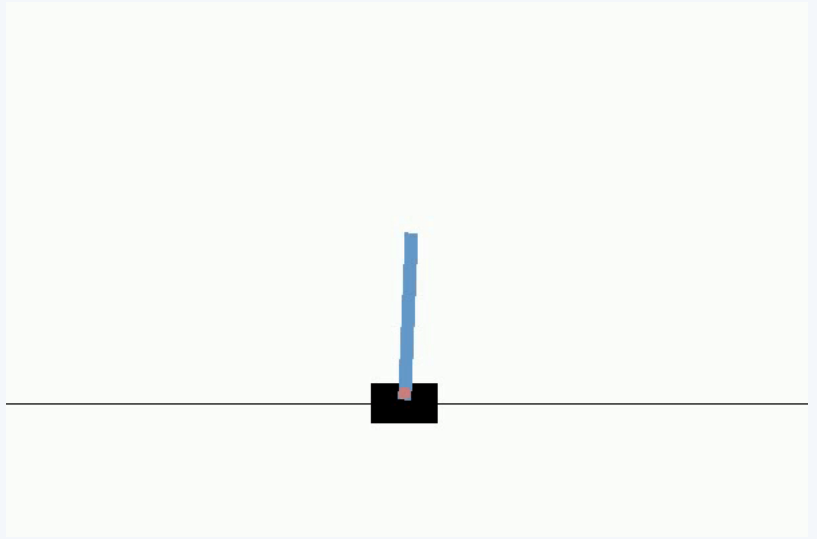
REINFORCEMENT LEARNING EXAMPLE

CartPole

An agent moves a cart left or right to keep a pole balanced.

RL FORMULATION

STATE	position, velocity, pole angle, angular velocity
ACTIONS	move left or move right
REWARD	remain balanced
OBJECTIVE	prevent the pole from falling



ENVIRONMENT

FULLY OBSERVABLE | SINGLE-AGENT | CONTINUOUS STATE | DISCRETE ACTIONS | EPISODIC

MAIN LESSON: States may be continuous even when actions are discrete.

REINFORCEMENT LEARNING EXAMPLE

Atari

An agent learns to play directly from images and game rewards (for example: Atari Breakout).

RL FORMULATION

OBSERVATION	pixels from the game screen
ACTIONS	joystick movements and buttons
REWARD	changes in game score
OBJECTIVE	achieve a high score



ENVIRONMENT

VISUAL INPUT | PARTIAL OBSERV. | SINGLE-AGENT | DISCRETE ACTIONS | SEQUENTIAL

MAIN LESSON: The agent may need to learn what to perceive as well as how to act.

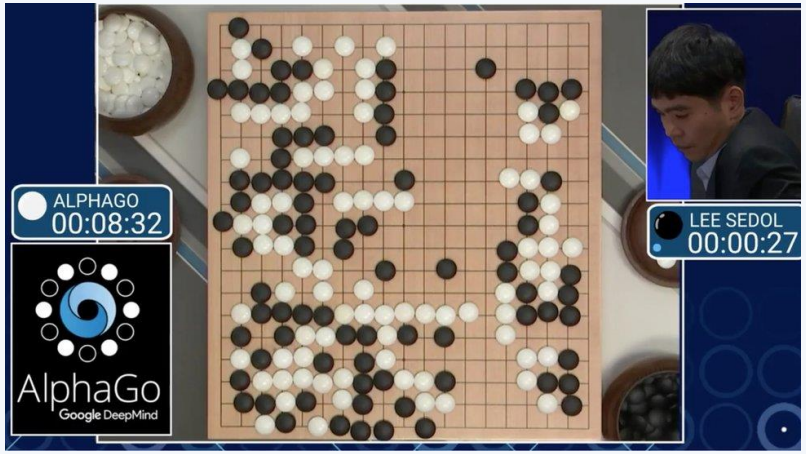
REINFORCEMENT LEARNING EXAMPLE

AlphaGo / AlphaZero

An agent improves through games, self-play, and look-ahead search.

RL FORMULATION

STATE	current board position
ACTIONS	legal moves
REWARD	final win or loss
OBJECTIVE	maximize the probability of winning



ENVIRONMENT

FULLY OBSERVABLE | TWO-AGENT | DETERMINISTIC | DISCRETE | LONG HORIZON

MAIN LESSON: Learning and planning can be combined.

REINFORCEMENT LEARNING EXAMPLE

Dota 2

A team of agents coordinates over a long match with incomplete information.

RL FORMULATION

OBSERVATION	partial game state
ACTIONS	movement, attacks, abilities, and tactical choices
REWARD	individual events and final match outcome
OBJECTIVE	cooperate to defeat the opposing team

DOTA 2



ENVIRONMENT

PARTIALLY OBSERVABLE | MULTI-AGENT | DYNAMIC | STRUCTURED ACTIONS | LONG HORIZON

MAIN LESSON: The consequence of an action may only become clear much later.

REINFORCEMENT LEARNING EXAMPLE

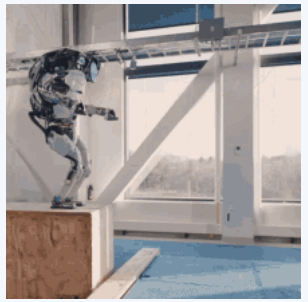
Robot Locomotion

A robot learns how to coordinate its joints to move stably.

RL FORMULATION

STATE	joint positions, velocities, orientation, contacts
ACTIONS	joint targets or motor commands
REWARD	progress, stability, smoothness, and efficiency
OBJECTIVE	move without falling or wasting energy





ENVIRONMENT

CONTINUOUS | DYNAMIC | STOCHASTIC | PARTIAL OBSERV. | SAFETY-CONSTRAINED

MAIN LESSON: The objective is specified, but the complete movement sequence is learned.

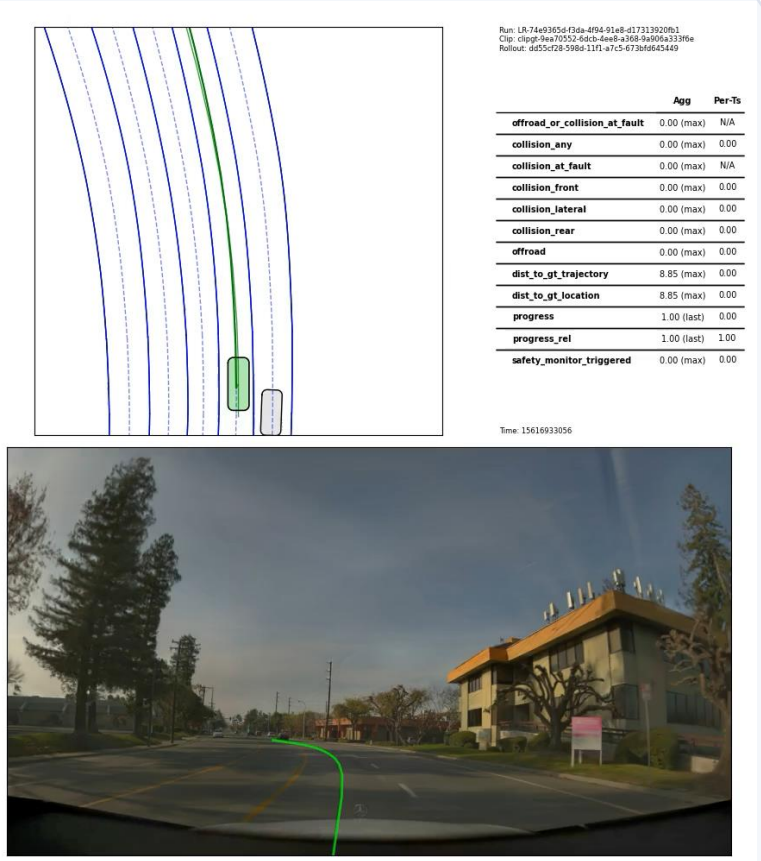
REINFORCEMENT LEARNING EXAMPLE

Autonomous Driving

An agent makes driving decisions while interacting with other road users.

RL FORMULATION

OBSERVATION	road, traffic, pedestrians, signals, and ego motion
ACTIONS	steering, acceleration, braking, or manoeuvres
REWARD	safety, progress, comfort, and rule compliance
OBJECTIVE	reach the destination safely and efficiently



ENVIRONMENT

PARTIAL OBSERV. | MULTI-AGENT | STOCHASTIC | HYBRID ACTIONS | SAFETY-CRITICAL

MAIN LESSON: Maximizing reward is not enough unless unsafe behaviour is prevented.

REINFORCEMENT LEARNING EXAMPLE

Autonomous Driving (Parking)

An agent makes driving decisions while interacting with other road users.

RL FORMULATION

OBSERVATION	road, traffic, pedestrians, signals, and ego motion
ACTIONS	steering, acceleration, braking, or manoeuvres
REWARD	safety, progress, comfort, and rule compliance
OBJECTIVE	reach the destination safely and efficiently



ENVIRONMENT

PARTIAL OBSERV. | MULTI-AGENT | STOCHASTIC | HYBRID ACTIONS | SAFETY-CRITICAL

MAIN LESSON: Maximizing reward is not enough unless unsafe behaviour is prevented.

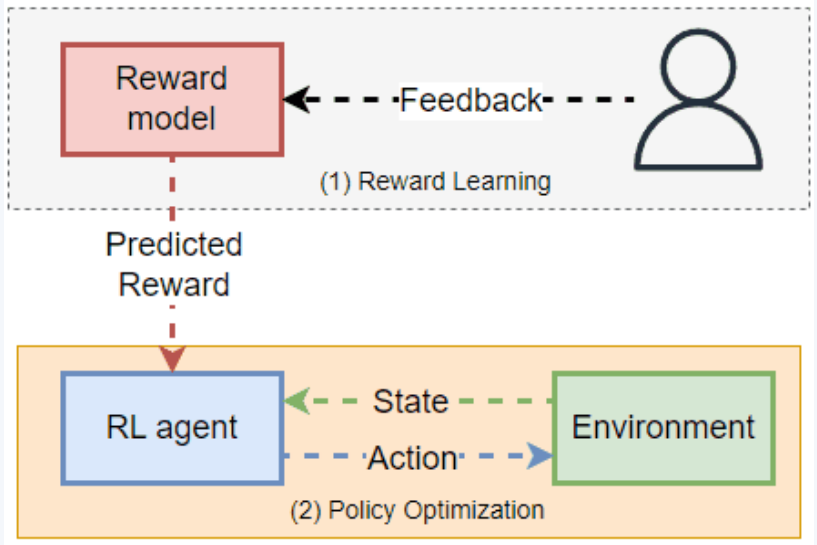
REINFORCEMENT LEARNING EXAMPLE

Language Model Alignment

A language model can be adapted using preferences about better and worse responses.

RL FORMULATION

STATE	conversation and generated text
ACTION	selecting the next token
REWARD	evaluation of the completed response
POLICY	probability distribution over possible tokens



ENVIRONMENT

SEQUENTIAL TEXT | LARGE ACTION SPACE | PREFERENCE REWARD | STOCHASTIC POLICY | NON-PHYSICAL

MAIN LESSON: An action can be a word, not only a physical movement.

Challenges in Reinforcement Learning

Unlike supervised learning's examples, RL learns from experience, and today that experience is usually simulated or tightly constrained.



1 Algorithmic improvement

Learn policies that are inherently transferable across domains, so behavior trained in simulation survives contact with the real world.

2 Simulation fidelity

Improve simulated physics and dynamics until virtual experience closely represents the physical systems where the agent will operate.

GOAL: Train in simulation, then deploy in reality with minimal retraining and predictable behavior.

What Makes Reinforcement Learning Difficult?

The agent must make decisions now without knowing their full future consequences.

CORE CHALLENGES

1	**FUTURE CONSEQUENCES**	Actions affect both the present and what happens next.
2	**DELAYED REWARDS**	Useful or harmful choices may only be revealed much later.
3	**PARTIAL OBSERVATION**	The agent may not see all information relevant to a decision.
4	**UNCERTAINTY**	The same action can lead to different outcomes.
5	**LEARNING FROM EXPERIENCE**	There is no teacher providing the correct action.

KEY QUESTION

How much of the past must the agent retain to predict what happens next?

The Gambler’s Fallacy

Independent events do not “remember” previous outcomes.

Five tails in a row do not make heads more likely on the next fair coin toss. The probability resets with every independent trial.

KEY QUESTION

How much of the past is actually needed to predict what happens next?



Five tails do not make the sixth toss “due.”

The Markov Property

Once the current state is known, earlier history adds no predictive information.

$$P(s_{t+1} \mid s_t, a_t, \text{earlier history}) = P(s_{t+1} \mid s_t, a_t)$$

The state must contain every feature that still matters for what happens next.

POLICY **$\pi(a \mid s)$**

The selected action depends on the current state.

Why the State Matters

A useful state is a compact summary of the relevant past.

INCOMPLETE STATE

ONE FRAME

Position is visible, but direction and speed are missing.



MARKOV STATE

FRAME STACK + MOTION

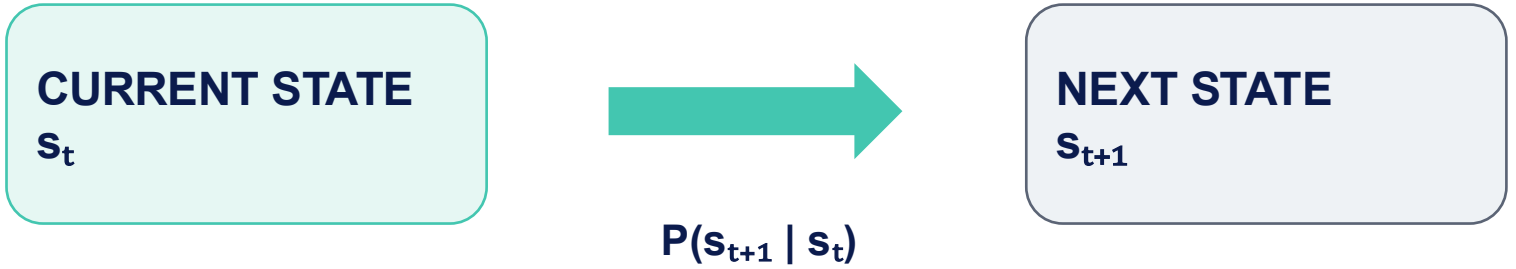
Position, direction, and speed together support prediction.

The Markov property depends on how the state is defined—not on the label “state.”

Markov Process

A Markov process describes how a system moves between states over time.

STATE TRANSITION

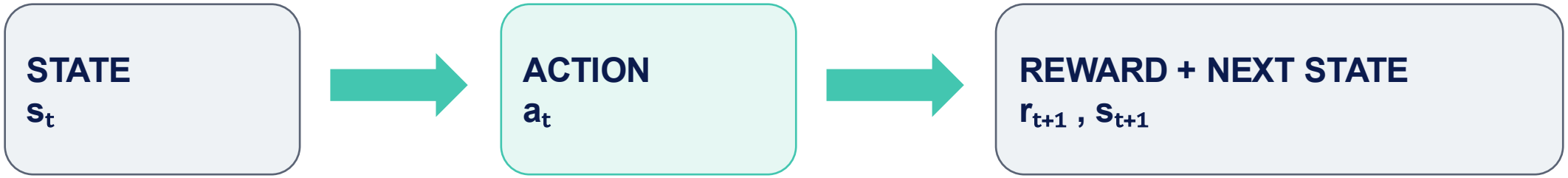


States S
Transition probabilities P
No agent yet

It explains how the world changes before an agent is introduced.

From a Process to a Decision Process

An MDP adds an agent whose choices can influence what happens next.



Process: what happens

Decision process: what happens after the agent chooses

Actions create control; rewards tell the agent which outcomes are desirable.

Markov Decision Process (MDP)

An MDP formally defines a sequential decision problem.

$$(S, A, P, R, \gamma)$$

problem specification

S	**States**	Where can the agent be?
A	**Actions**	What can the agent do?
P	**Transitions**	What may happen next?
R	**Rewards**	How is the outcome evaluated?
γ	**Discount**	How much do future rewards matter?

Where Reinforcement Learning Enters

The environment defines the problem; learning improves the agent's behaviour.

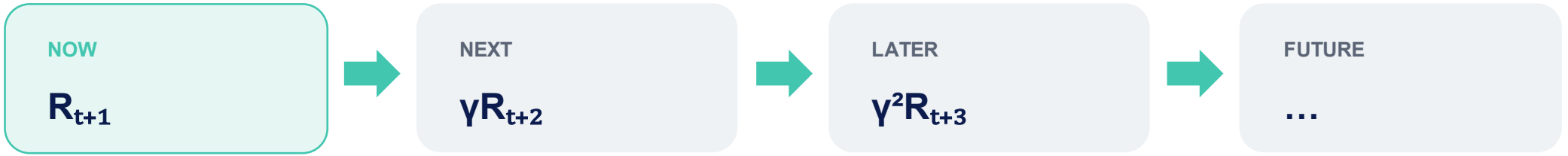
1	**MDP**	Defines states, actions, transitions, and rewards.
2	**POLICY**	Maps the current state to an action: $\pi(a \mid s)$
3	**LEARNING**	Uses experience to improve the policy for long-term reward.

Immediate Reward or Long-Term Outcome

A good action may sacrifice reward now to create a better future.

RETURN

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$



Example: in Grid World, moving away from the goal may be necessary to go around a wall.

Bellman Idea (Equation)

The value of the present combines reward now with the value of what follows.

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[R_{t+1} + \gamma\, V^{\pi}(S_{t+1}) \mid S_t = s \right]$$

VALUE NOW
V(s)

=

REWARD NOW
R

+

VALUE LATER
discount × V(s′)

Future value flows backward, allowing earlier states and actions to inherit value from later outcomes.

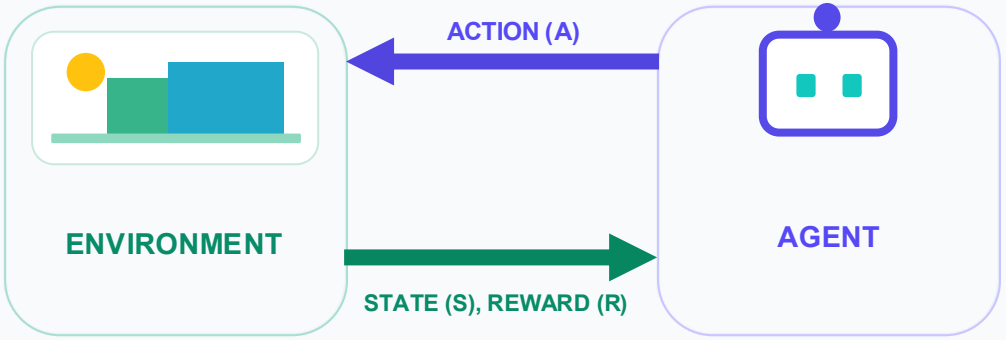
Review

The logic builds from prediction to control to learning.

1	**INDEPENDENCE**	Past outcomes do not change an independent next event.
2	**MARKOV STATE**	The present summarizes all relevant history.
3	**MARKOV PROCESS**	Transition probabilities describe how states change.
4	**MDP**	Actions and rewards turn prediction into a decision problem.
5	**REINFORCEMENT LEARNING**	Experience improves the policy for long-term reward.

Core idea: choose actions using a state that preserves what matters for the future.

Reinforcement Learning



ENVIRONMENT

- R (REWARD)
- S (STATE)
- P (TRANSITION DYNAMICS)

AGENT

- A (ACTION)
- π (POLICY) state → action rule
- γ (DISCOUNT FACTOR)

The Reinforcement Learning Loop

One interaction step produces one transition.

OBSERVE
current state s_t

→

CHOOSE
a_t using $\pi(a \mid s)$

→

TRANSITION
environment → s_{t+1}

→

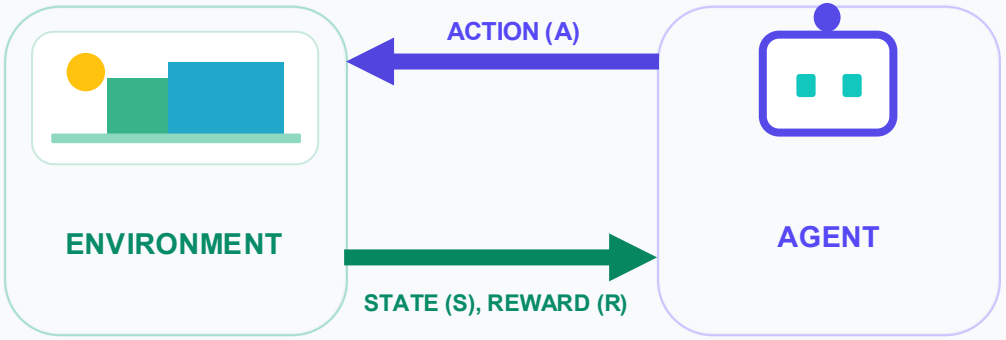
REWARD
receive r_{t+1}

→

LEARN + REPEAT
use $(s_t, a_t, r_{t+1}, s_{t+1})$

Observe → act → transition → reward → learn → repeat from s_{t+1}

Reinforcement Learning



ENVIRONMENT

- R (REWARD)
- S (STATE)
- P (TRANSITION DYNAMICS)

AGENT

- A (ACTION)
- π (POLICY) state → action rule
- γ (DISCOUNT FACTOR)

Timesteps, Trajectories, and Episodes

The same loop can run for one step, a finite episode, or indefinitely.

LOOP TYPES

EPISODIC
Natural ending • terminal state s_T
Examples: goal, checkmate, game over

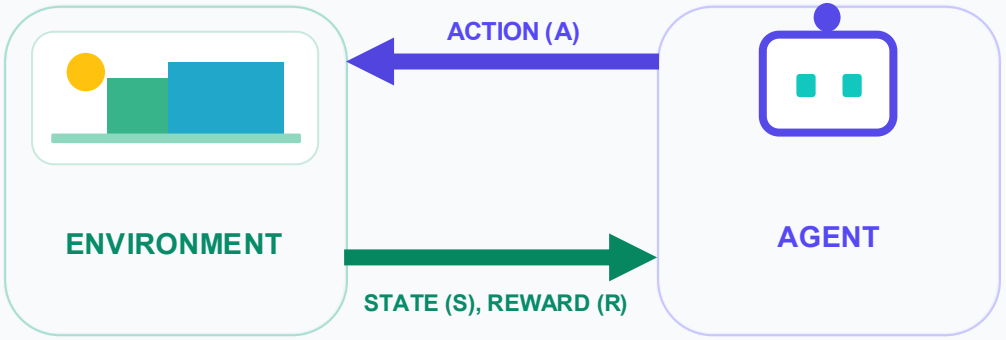
CONTINUING
No natural final state
Examples: temperature or traffic control

LOOP UNITS

TIMESTEP	one pass through the loop
TRAJECTORY	a recorded sequence of transitions
EPISODE	start-to-termination trajectory
ROLLOUT	any recorded run or segment

A rollout may be a complete episode—or only a short segment of a continuing task.

Reinforcement Learning



ENVIRONMENT

- R (REWARD)
- S (STATE)
- P (TRANSITION DYNAMICS)

AGENT

- A (ACTION)
- π (POLICY) state → action rule
- γ (DISCOUNT FACTOR)

The Policy

The policy is the agent's decision rule.

$$\pi(a \mid s)$$

state in → action probabilities out

DECISION FLOW

INPUT
current state s

→

OUTPUT
distribution over actions

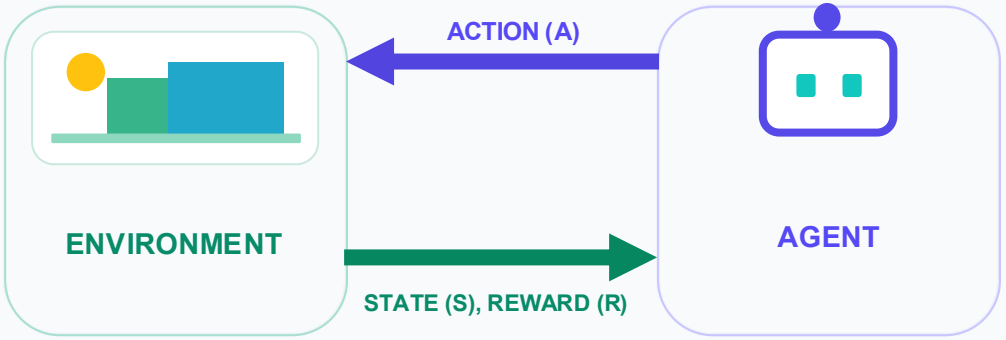
→

DECISION
sample action a

POSSIBLE IMPLEMENTATIONS

Lookup table	Linear model	MLP / CNN	RNN / LSTM	Transformer

Reinforcement Learning



ENVIRONMENT

- R (REWARD)
- S (STATE)
- P (TRANSITION DYNAMICS)

AGENT

- A (ACTION)
- π (POLICY) state → action rule
- γ (DISCOUNT FACTOR)

Policy vs. Transition Dynamics

Two probability distributions answer two different questions.

AGENT

$\pi(a \mid s)$

Which action does the agent choose?
Describes behaviour • learned or controlled by the agent

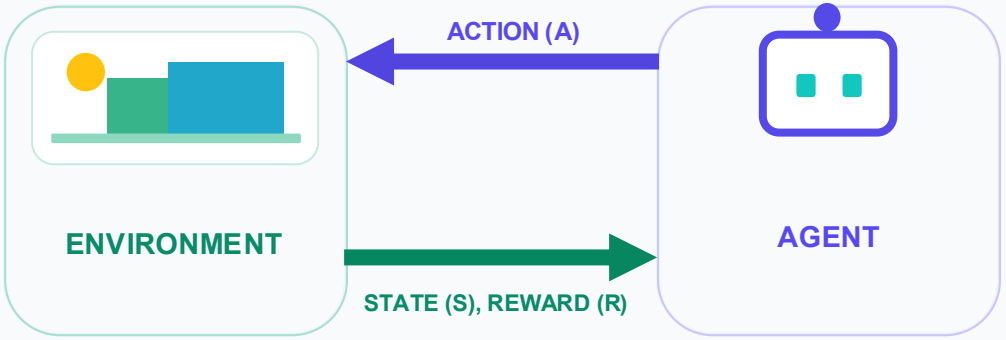
ENVIRONMENT

$P(s' \mid s, a)$

What happens after the action?
Describes the rules or physics of the world

MODEL-BASED RL The agent also learns an internal model of the transition dynamics.

Reinforcement Learning



ENVIRONMENT

- R (REWARD)
- S (STATE)
- P (TRANSITION DYNAMICS)

AGENT

- A (ACTION)
- π (POLICY) state → action rule
- γ (DISCOUNT FACTOR)

State and Action Spaces

The spaces stay fixed; the current state and chosen action change.

STATE SPACE S

$s_t \in S$

All possible situations
Discrete: grid cells • Continuous: joint values

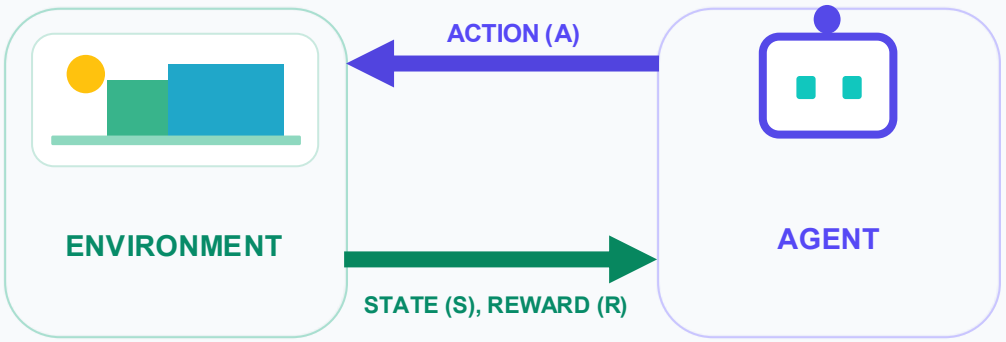
ACTION SPACE A

$a_t \in A$

All possible choices
Discrete: left/right • Continuous: steering angle

At each timestep: the environment occupies one state; the agent selects one action.

Reinforcement Learning



ENVIRONMENT

- R (REWARD)
- S (STATE)
- P (TRANSITION DYNAMICS)

AGENT

- A (ACTION)
- π (POLICY) state → action rule
- γ (DISCOUNT FACTOR)

Reward

Immediate feedback from the environment about one transition.

$$r_{t+1} = R(s_t, a_t, s_{t+1})$$

REWARD PROPERTIES

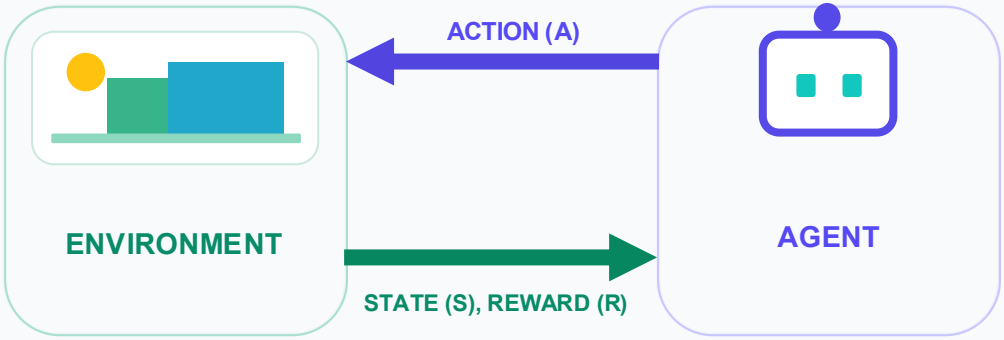
- one scalar value
- produced by the environment
- positive, negative, or zero
- encodes what the task encourages

REWARD evaluates one transition

RETURN evaluates long-term consequences

Reward is the feedback signal—not the final objective.

Reinforcement Learning



ENVIRONMENT

- R (REWARD)
- S (STATE)
- P (TRANSITION DYNAMICS)

AGENT

- A (ACTION)
- π (POLICY) state → action rule
- γ (DISCOUNT FACTOR)

Return and the Discount Factor

Return combines local rewards into a measure of long-term success.

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots$$

DISCOUNT FACTOR $\gamma \in [0, 1]$

SMALLER γ — near-term rewards dominate — future rewards remain important — LARGER γ

RL OBJECTIVE maximize expected return

3 Types of Reinforcement Learning

Different RL algorithm families learn and act in fundamentally different ways.

Model-based

- Learn a model of the world, then plan using the model
- Update the model often
- Re-plan often

Value-based

- Learn the state or state-action value
- Act by choosing the best action in a state
- Exploration is a necessary add-on

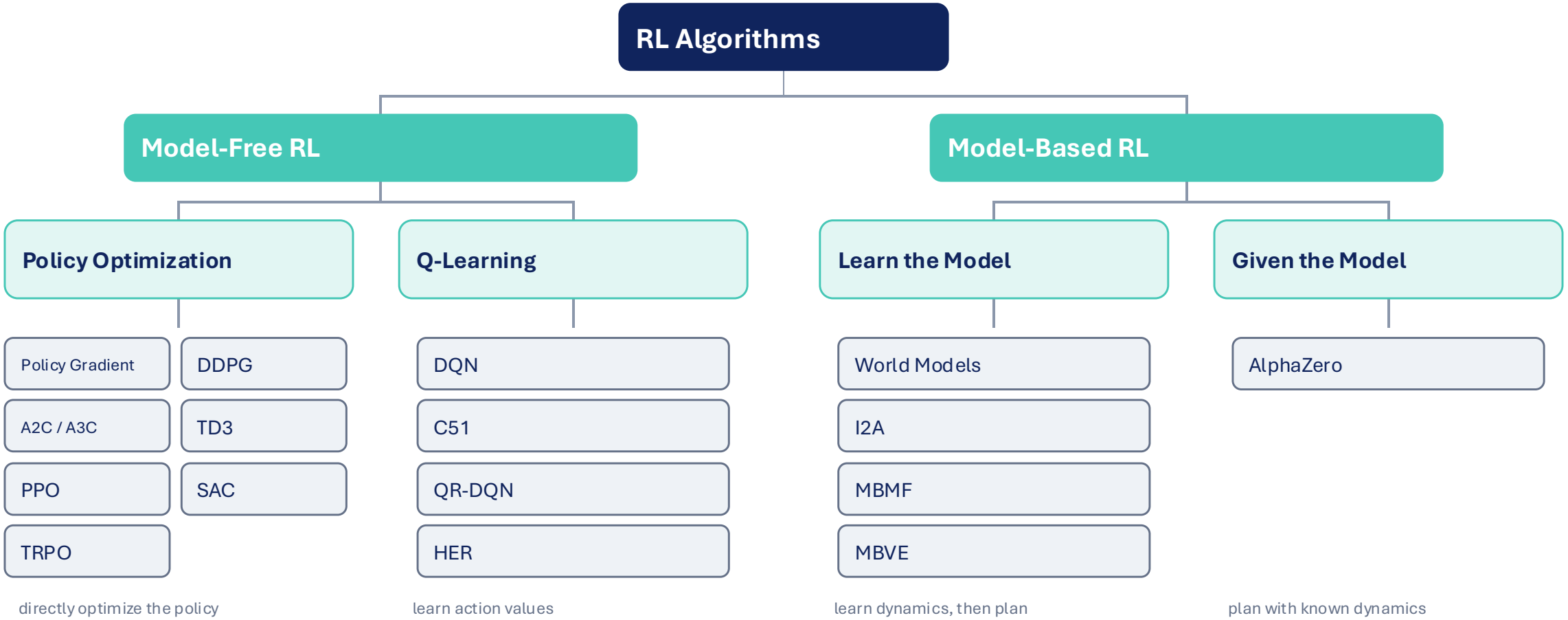
Policy-based

- Learn the stochastic policy function mapping state to action
- Act by sampling the policy
- Exploration is baked in

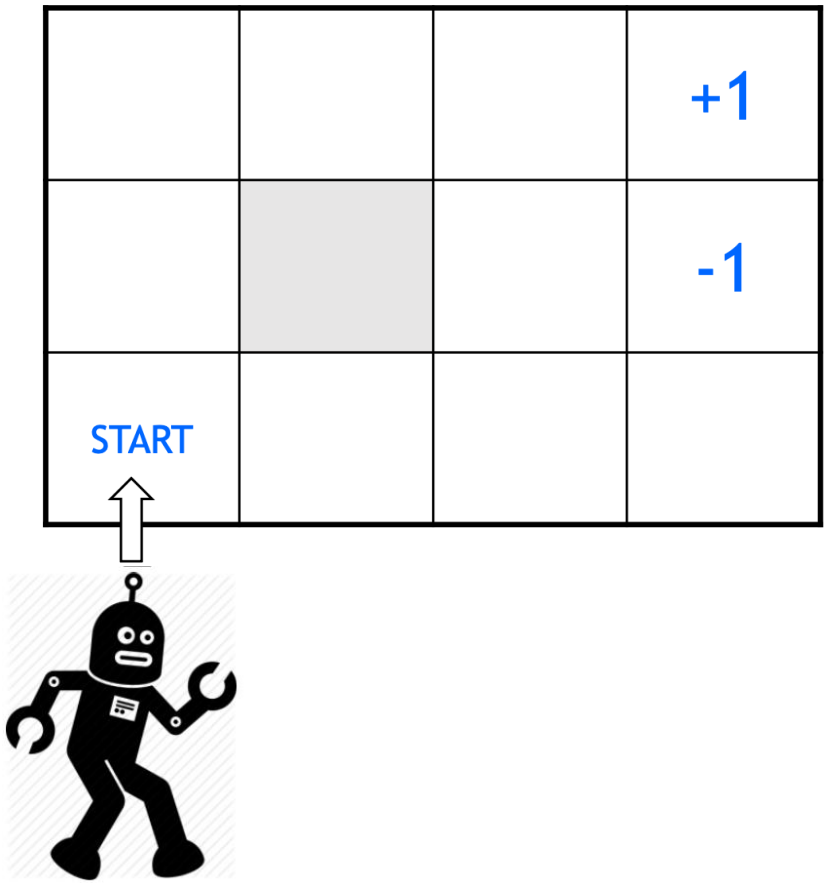
These families can be combined. Many state-of-the-art algorithms blend value- and policy-based learning.

Taxonomy of RL Methods

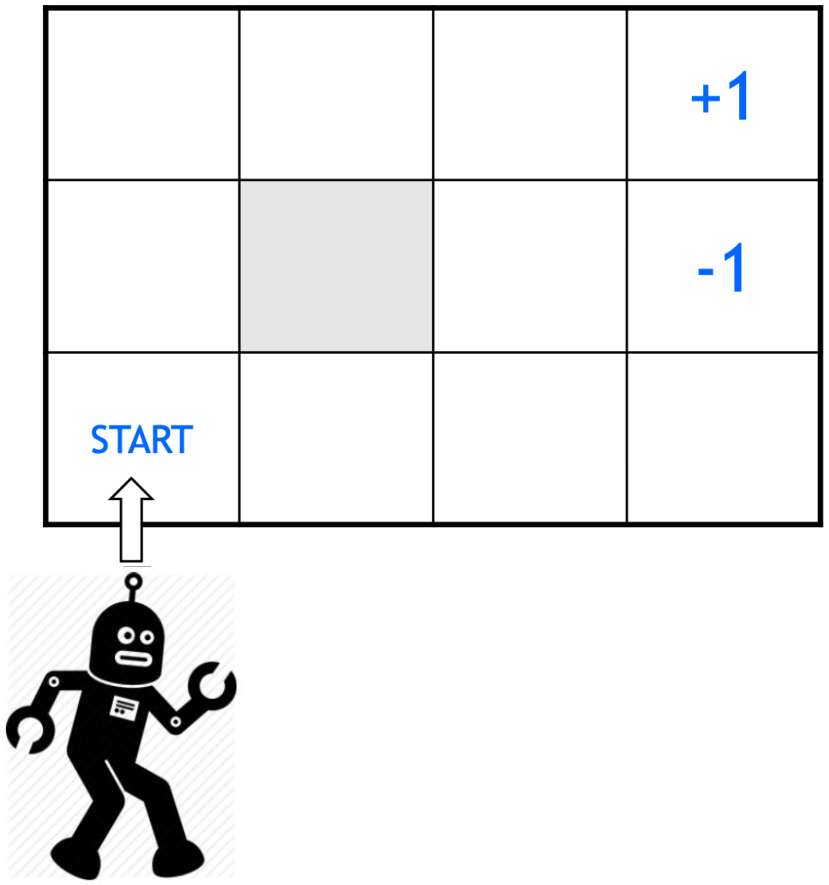
The model-free vs. model-based split is the first fork in choosing an RL algorithm.



Grid World: An example Problem



Grid World: Defining the Problem

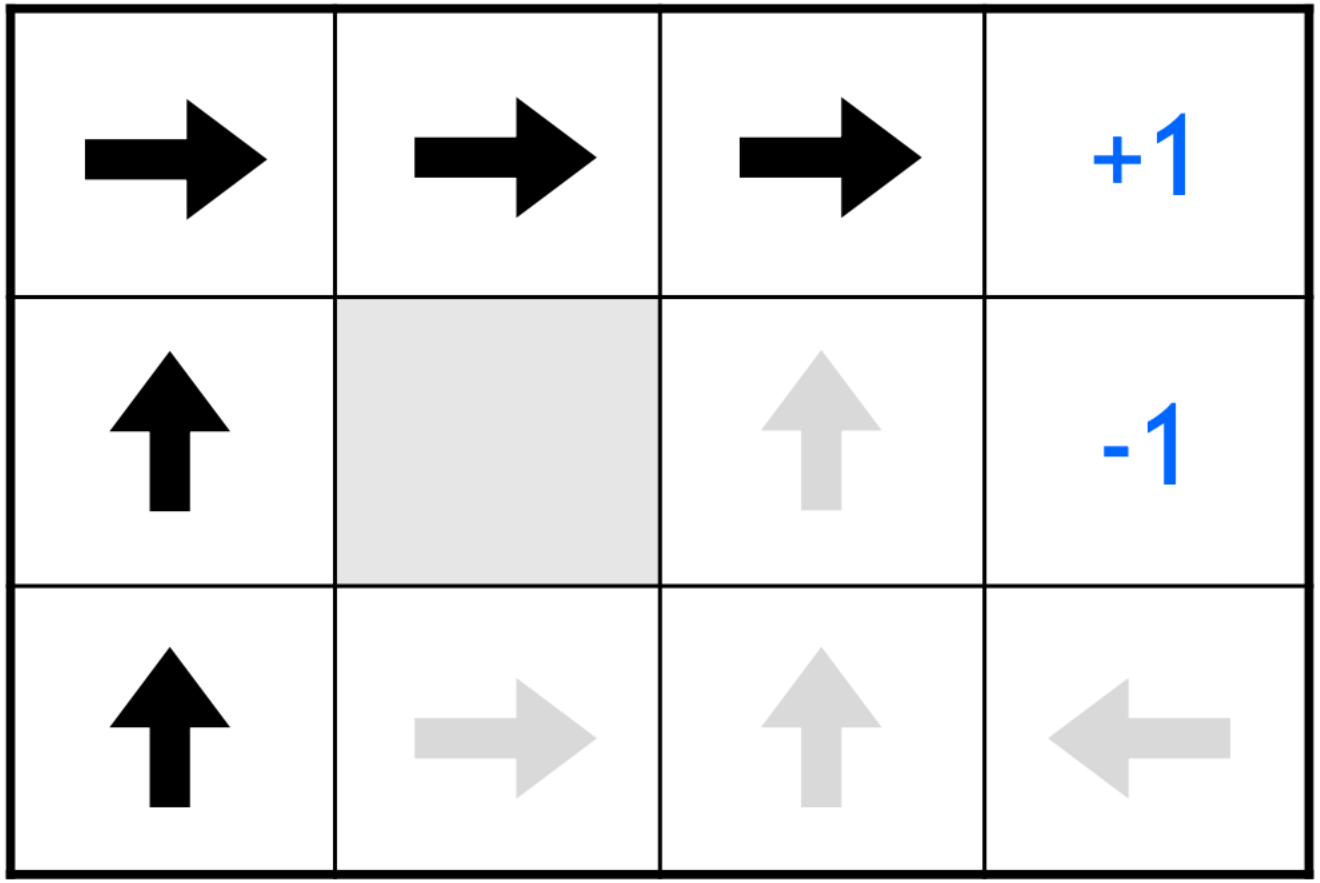


- **Actions:** UP, DOWN, LEFT, RIGHT
- **Stochastic transitions:** 80% intended direction, 10% left, 10% right
- **Reward:** +1 at goal, –1 at pit, –0.04 per step
- **Goal:** find the policy that maximizes total reward

Three approaches: learn a model & plan, learn action values, or learn the policy directly.

Optimal Policy: Deterministic World

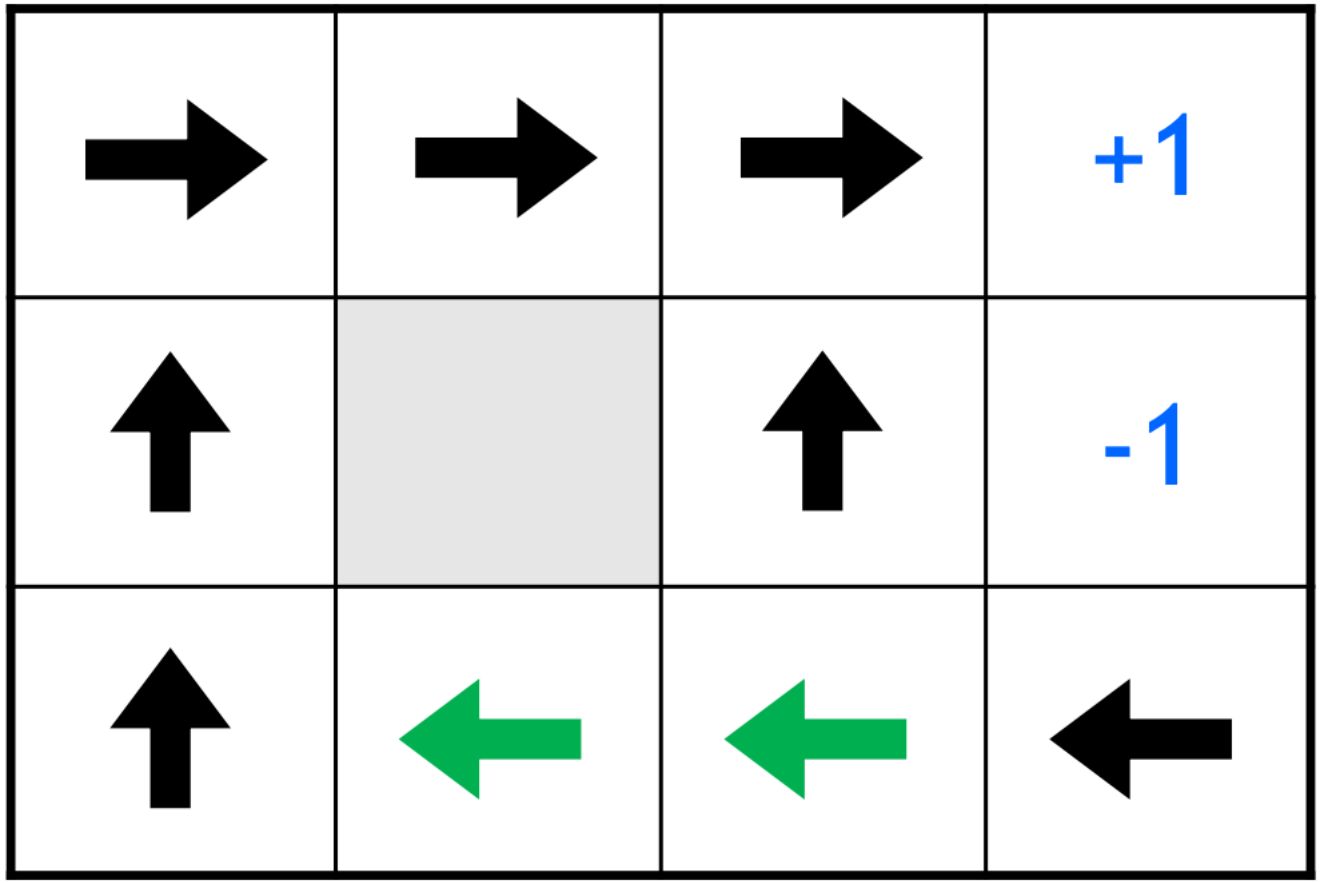
Reward: –0.04 per step — actions move exactly as intended (100% success)



Policy: shortest path to the goal

Optimal Policy: Stochastic World

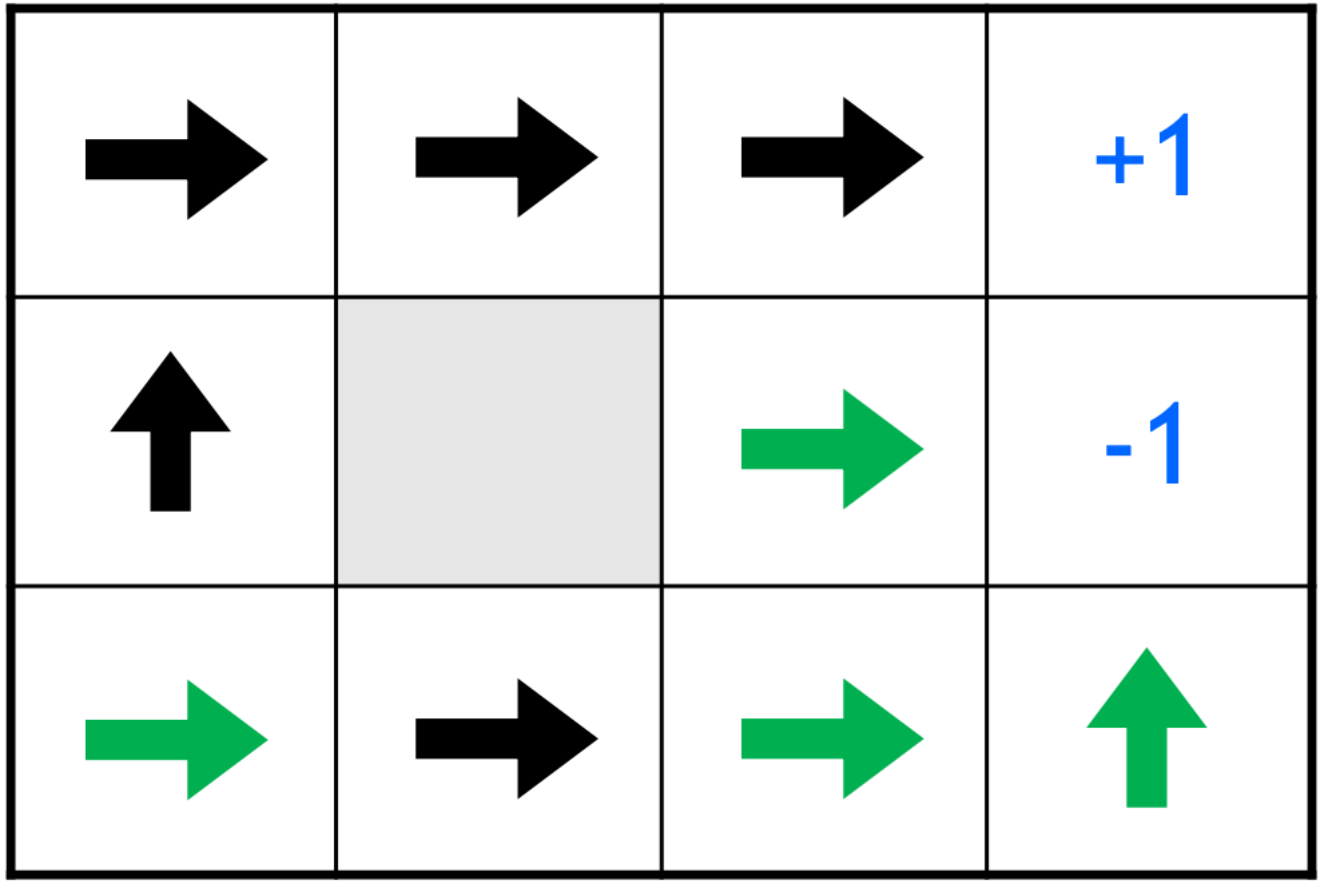
Reward: –0.04 per step — actions succeed only 80% of the time (10% left, 10% right)



Policy: shortest path, steering clear of the –1 pit

Optimal Policy: Higher Step Penalty

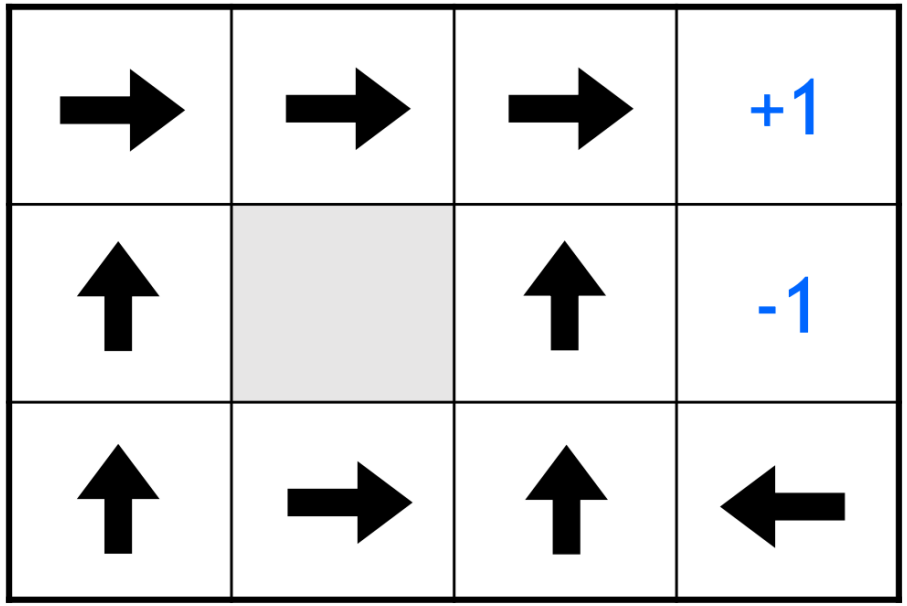
Reward: –2 per step — same stochastic actions (80% intended, 10% left, 10% right)



Policy: shortest path — the large step penalty outweighs the risk near the pit

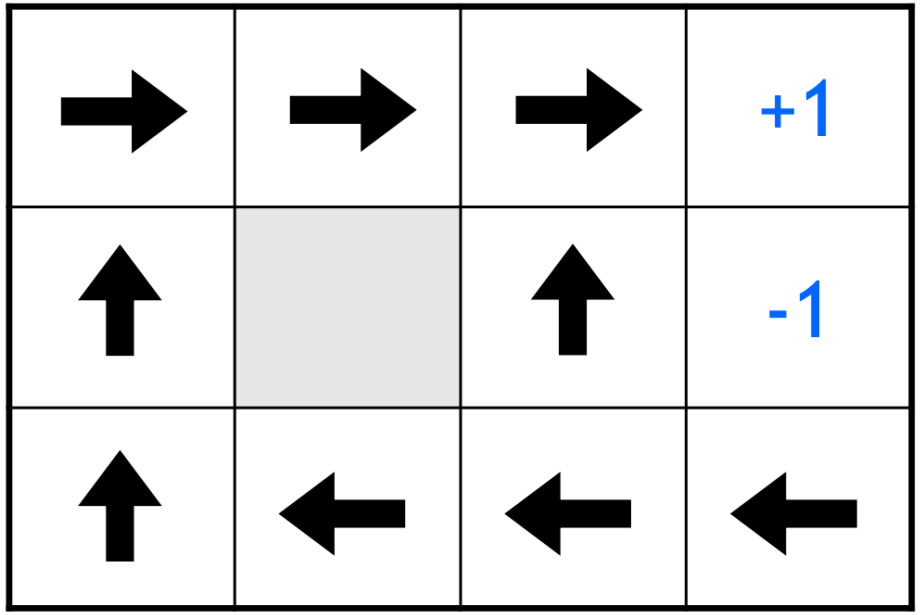
Optimal Policy: Effect of Step Penalty Magnitude

Reward: –0.1 per step



More urgent — hurry to the goal

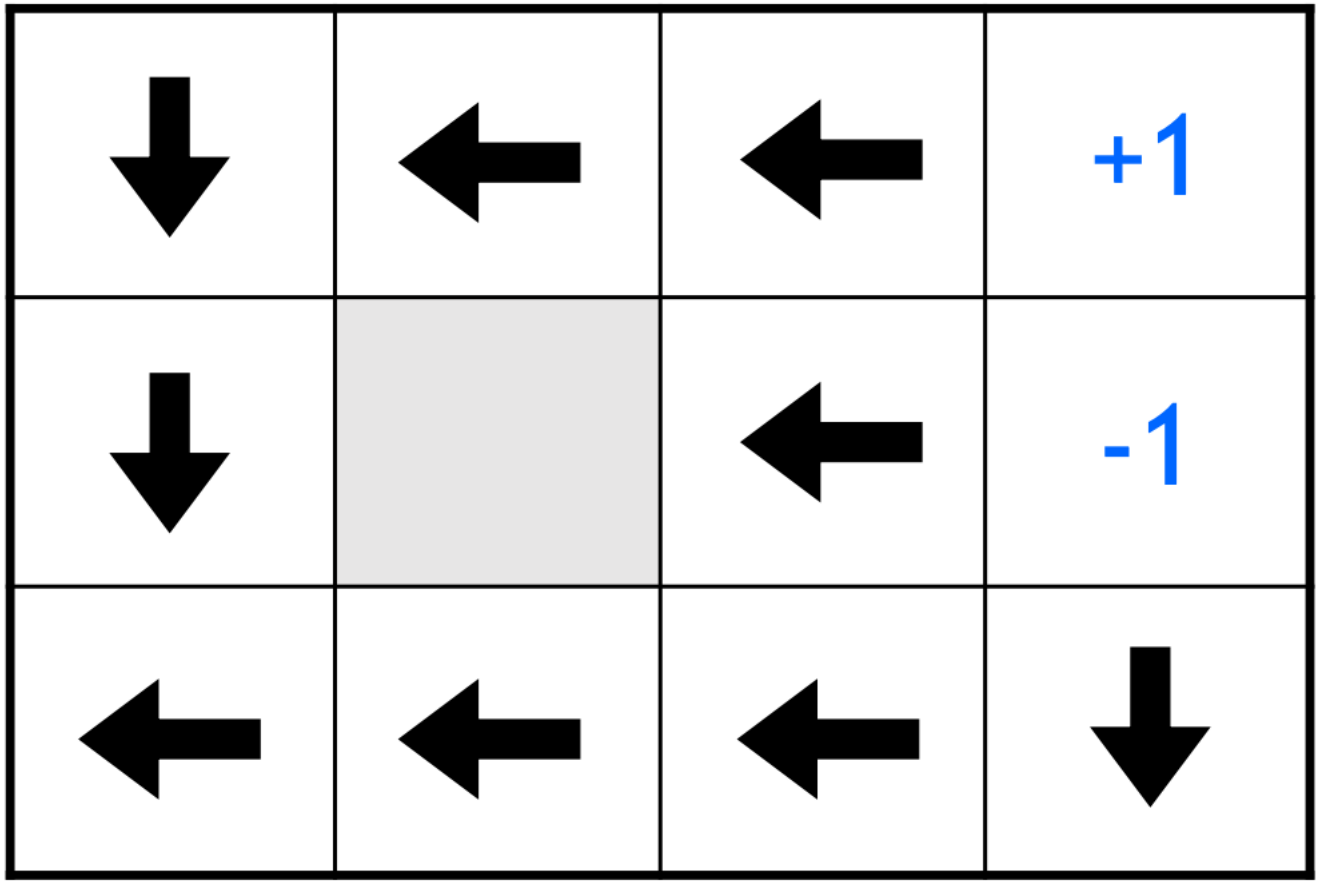
Reward: –0.04 per step



Less urgent — can afford a detour

Optimal Policy: Positive Step Reward

Reward: +0.01 per step — same stochastic actions (80% intended, 10% left, 10% right)



Policy: longest path — wandering earns more than finishing